Correction TP1

L'objectif de ce TP vise à construire un modèle de réseau de neurones pour classer les images de vêtements en utilisant la base de données **`fashion_mnist`**. Cette dernière représente un ensemble de données d'images d'articles de Zalando composé d'un ensemble d'apprentissage de **`60000`** exemples et d'un ensemble de test de **`10000`** exemples. Chaque exemple est une image en niveaux de gris **`28×28`**, associée à une **étiquette** de **`10`** **classes** (**`0 : T-shirt/top, 1 : Trouser, 2 : Pullover, 3 : Dress, 4 : Coat, 5 : Sandal, 6 : Shirt, 7 : Sneaker, 8 : Bag, 9 : Ankle boot`**)

NB. Lancer Google Colab et téléchargez le fichier **`TP1_DL.ipynb`**. Ensuite, exécuter la première cellule du notebook qui vise à l'importation des bibliothèques.

```
import tensorflow as tf

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix,classification_report, accuracy_score

print(tf.__version__)
```

Correction TP1 : 1/

1/ Importer la base de données **`fashion_mnist`** à partir du package **`datasets`** de **`tf.keras`**, puis charger les données d'apprentissage et de test dans **4** tableaux de type **ndarray** **`train_images`**, **`train_labels`**, **`test_images`**, et **`test_labels`** en utilisant ***`load_data`***

```
train_data, test_data = tf.keras.datasets.fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
train_images, train_labels=train_data
```

```
test_images, test_labels=test_data
```

Correction TP1 : 2/

2/ Afficher la taille des tableaux **`train_images`**, **`train_labels`**, **`test_images`**, et **`test_labels`**

```
print("train_images.shape=", train_images.shape)
```

```
train_images.shape= (60000, 28, 28)
```

```
print("train_labels.shape=", train_labels.shape)
```

```
train_labels.shape= (60000,)
```

```
print("test_images.shape=", test_images.shape)
```

```
test_images.shape= (10000, 28, 28)
```

```
print("test_labels.shape=", test_labels.shape)
```

```
test_labels.shape= (10000,)
```

Correction TP1 : 3/

3/ Normaliser les valeurs des pixels de chaque image dans **`train_images`**, et **`test_images`** de sorte qu'on met les valeurs dans la même échelle de `0` à `1`

```
train_images = train_images / 255.0

test_images = test_images / 255.0
```

Ou bien

```
train_images, test_images = train_images / 255.0, test_images / 255.0
```

Correction TP1 : 4/

4/ Décomposer les données d'apprentissage en deux parties : **90%** sélectionnées aléatoirement sans répétions pour l'apprentissage (**`train_images`**, **`train_labels`**), et le reste pour la validation **`val_images`**, **`val_labels`**) (utiliser ***`train_test_split`*** de ***`sklearn.model_selection`***) Afficher la taille des tableaux **`train_images`** et **`val_images`**

```
train_images, val_images, train_labels, val_labels =

train_test_split(train_images, train_labels, test_size=0.1, random_state=1)
```

```
print(train_images.shape, train_labels.shape, val_images.shape, val_labels.shape)
```

```
(54000, 28, 28) (54000,) (6000, 28, 28) (6000,)
```

Correction TP1 : 5/

5/ Afficher les **12** premières images de l'ensemble d'apprentissage (Spécifier le nom de la classe sous chaque image)

```
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
               'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']
```




```
plt.figure(figsize=(4,4))
for i in range(12):
    plt.subplot(3,4,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(train_images[i], cmap=plt.cm.gray)
    plt.xlabel(class_names[train_labels[i]])
plt.show()
```

Correction TP1 : 6/

6/ Construire un modèle de prédiction, nommé **`cls_fashion`**, en appliquant un réseau de neurones de **`5`** **couches** (une couche d'entrée, **`3`** couches cachées, et une couche de sortie). La **couche d'entrée** transforme le format des images d'un tableau à deux dimensions (**`28×28`** pixels) en un tableau à une dimension. Les deux premières **couches cachées** comportent **`128`** neurones tandis que la troisième couche cachée comporte **`64`** neurones.

Afficher un **résumé** de l'architecture de réseau de neurones de **`cls_fashion`**

NB.

- Activer les neurones des couches cachées par la fonction **`ReLu`**
- Configurer les couches de **`cls_fashion`** en utilisant la classe **`Sequential`** de **`tf.keras`**, la classe **`Flatten`** de **`tf.keras.layers`** pour configurer la couche d'entrée, et la classe **`Dense`** de **`tf.keras.layers`** pour configurer les couches cachées et de sortie

Correction TP1 : 6-2/

```
#construction du modèle
cls_fashion = tf.keras.Sequential([
      #Couche d'entrée
      tf.keras.layers.Input(shape=(28, 28)),#forme d'entrée
      tf.keras.layers.Flatten(), # Appliquer Flatten
      #Première couche cachée
      tf.keras.layers.Dense(128, activation="relu"),
     #Deuxième couche cachée
     tf.keras.layers.Dense(128, activation="relu"),
      #Troisième couche cachée
      tf.keras.layers.Dense(64, activation="relu"),
     #Couche de sortie
      tf.keras.layers.Dense(10, activation="softmax")
])
```

Correction TP1 : 6-3/

```
cls_fashion.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 128)	16,512
dense_2 (Dense)	(None, 64)	8,256
dense_3 (Dense)	(None, 10)	650

Total params: 125,898 (491.79 KB)
Trainable params: 125,898 (491.79 KB)
Non-trainable params: 0 (0.00 B)

Correction TP1 : 6-4/

```
print(cls_fashion.get_weights()[0].shape)
print(cls_fashion.get_weights()[1].shape)
print(cls_fashion.get_weights()[2].shape)
print(cls_fashion.get_weights()[3].shape)
```

```
(784, 128)
(128,)
(128, 128)
(128,)
```

Correction TP1 : 7/

7/ Compiler le modèle **`cls_fashion`** en appliquant l'**optimiseur** **`"adam"`** et entraîner le pour un **nombre d'époques** égal à **`10`**.

```
#Compilation du modèle
cls_fashion.compile(optimizer='adam',
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])
#Apprentissage
history=cls_fashion.fit(train_images, train_labels,
                        batch_size=32, epochs=10,
                        validation_data=(val_images, val_labels))
```

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 4ms/step - accuracy: 0.7593 - loss: 0.6648 - val_accuracy: 0.8580 - val_loss: 0.3942
```

```
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9094 - loss: 0.2365 - val_accuracy: 0.8775 - val_loss: 0.3502
```

Correction TP1 : 8/

8/ Représenter la courbe d'évaluation de **`cls_fashion`** en fonction du nombre époques sur les données d'**apprentissage** et les données de **validation**

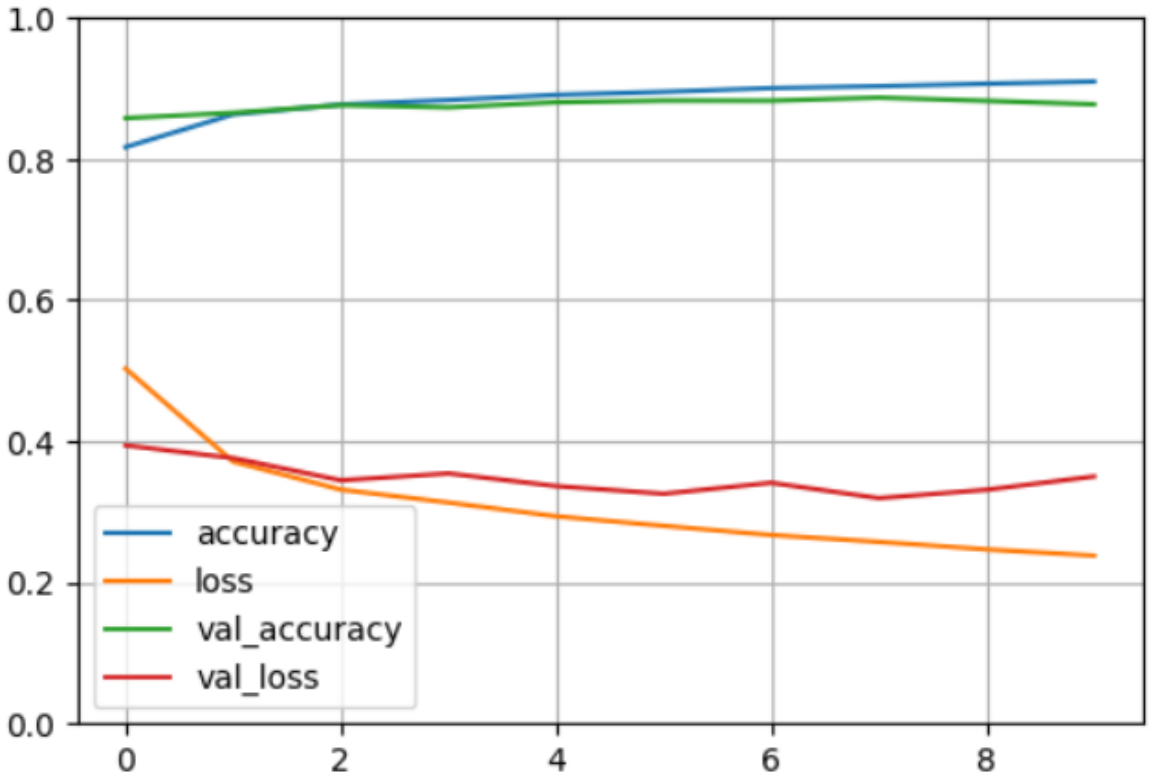
`history.history`


```
{'accuracy': [0.816944420337677,
  0.8632962703704834,
  0.8771851658821106,
  0.8836110830307007,
  0.8908703923225403,
  0.895203709602356,
  0.9004629850387573,
  0.9032407402992249,
  0.9067037105560303,
  0.9099074006080627],
 'loss': [0.5029152631759644,
  0.37116435170173645,
  0.3315637707710266,
  0.3132384121417999,
  0.29377561807632446,
  0.2802329957485199,
  0.2672460675239563,
  0.25756964087486267,
  0.24688772857189178,
  0.23799774050712585],
 'val_accuracy': [0.8579999804496765,
  0.8654999732971191,
  0.8763333559036255,
  0.8731666803359985,
  0.8805000185966492,
  0.8828333616256714,
  0.8826666474342346,
  0.8870000243186951,
  0.8824999928474426,
  0.8774999976158142],
 'val_loss': [0.39424437284469604,
  0.37620726227760315,
  0.3447675406932831,
  0.3544919490814209,
  0.3366203308105469,
  0.32573747634887695,
  0.3412637412548065,
  0.3195032775402069,
  0.331374853849411,
  0.35020411014556885]}
```

Correction TP1 : 8-2/

8/ Représenter la courbe d'évaluation de **`cls_fashion`** en fonction du nombre époques sur les données d'**apprentissage** et les données de **validation**

```
pd.DataFrame(history.history).plot(figsize=(6, 4))
plt.grid(True)
plt.gca().set_ylim(0, 1) # set the vertical range to [0-1]
plt.show()
```



Correction TP1 : 9/

9/ Représenter la **matrice de confusion** associée aux données de test selon **`cls_fashion`**

```
# Forme des images de test
test_images.shape
```

```
(10000, 28, 28)
```

```
# Extraction d'une image
test_images[0:1].shape
```

```
(1, 28, 28)
```

```
# Calcul des probalibités d'appratenance
# de cette image à toutes les classes
pred_0=cls_fashion.predict(test_images[0:1])
pred_0
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 719ms/step
array([[2.8752135e-07, 6.6754525e-05, 4.2202223e-06, 2.8260426e-06,
        1.1503624e-05, 6.7358143e-03, 6.0076115e-07, 6.9575533e-03,
        3.3703679e-06, 9.8621708e-01]], dtype=float32)
```

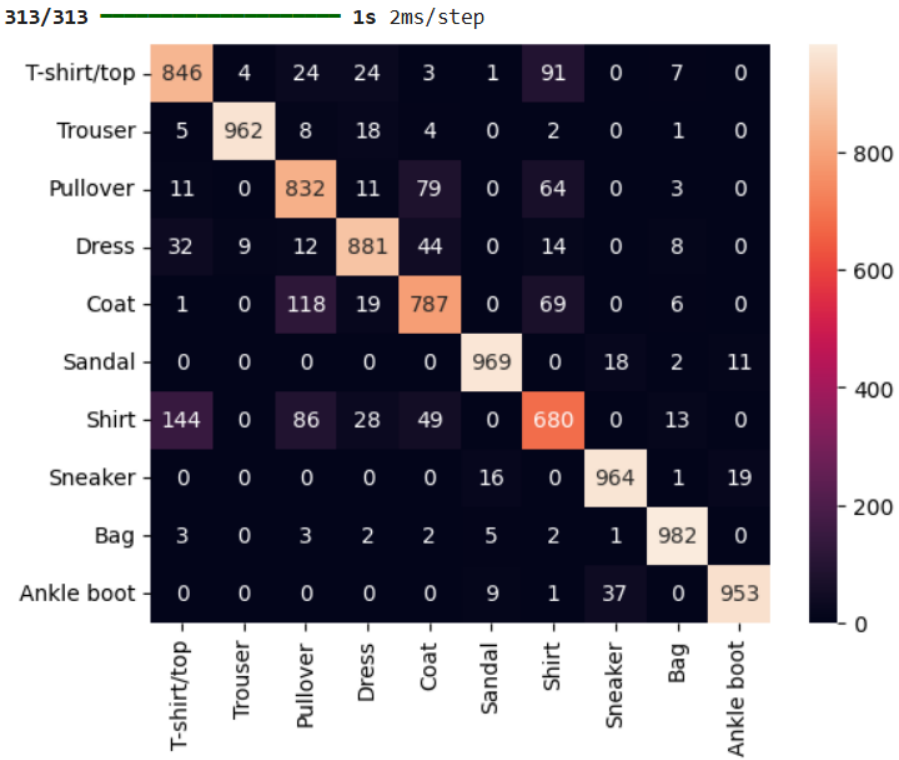
```
# Trouver la classe dont la probabité est maximale
class_0 = np.argmax(pred_0, axis=1)
class_0
```

```
array([9])
```

Correction TP1 : 9-2/

9/ Représenter la **matrice de confusion** associée aux données de test selon **`cls_fashion`**

```
pred_labels=cls_fashion.predict(test_images)
pred_labels = np.argmax(pred_labels, axis=1)
cm = confusion_matrix(test_labels, pred_labels)
sns.heatmap(cm, annot=True, fmt='d',
            xticklabels=class_names, yticklabels=class_names)
plt.show()
```



Correction TP1 : 10/

10/ Calculer l'exactitude (**accuracy**) et la perte (**loss**) associées aux données de test selon **`cls_fashion`**

```
loss_t, acc_t = cls_fashion.evaluate(test_images, test_labels)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8870 - loss: 0.3301
```

Correction TP1 : 11/

11/ Déduire le **rapport de classification** associée aux données de test selon **`cls_fashion`**

```
print(classification_report(test_labels, pred_labels, target_names= class_names))
```

```
                precision    recall  f1-score   support

   T-shirt/top       0.81      0.85      0.83      1000
       Trouser       0.99      0.96      0.97      1000
      Pullover       0.77      0.83      0.80      1000
         Dress       0.90      0.88      0.89      1000
          Coat       0.81      0.79      0.80      1000
        Sandal       0.97      0.97      0.97      1000
         Shirt       0.74      0.68      0.71      1000
       Sneaker       0.95      0.96      0.95      1000
           Bag       0.96      0.98      0.97      1000
    Ankle boot       0.97      0.95      0.96      1000

      accuracy                           0.89     10000
     macro avg       0.89      0.89      0.89     10000
  weighted avg       0.89      0.89      0.89     10000
```